

Integrating A* Pathfinding and Virtual Wall Guidance for Left Wall-Following on iRobot Create 3

1st Soheil Behnam Roudsari

Smart Systems Engineering
Hanze University of Applied Sciences
Assen, The Netherlands
s.behnam.roudsari@st.hanze.nl

2nd Rianne Drijver

Smart Systems Engineering
Hanze University of Applied Sciences
Assen, The Netherlands
r.drijver@st.hanze.nl

3rd Amin Md Khandoker Ruhul

Smart Systems Engineering
Hanze University of Applied Sciences
Assen, The Netherlands
mkr.amin@st.hanze.nl

Abstract—This report explores the challenge of autonomous robot navigation in a maze. The main goal is to help the robot navigate smoothly and reach a set target while avoiding obstacles. The solution combines image processing, the A* pathfinding algorithm, and real-time sensor data. Image transformations and edge detection are used to map the maze. Virtual wall method was also introduced, where the robot's sensors are adjusted to keep it on the path by simulating obstacles near the edges. The results show the robot successfully following the path, finding the goal, and celebrating upon reaching it. The conclusion highlights that this approach is effective for reliable robot navigation, with potential uses in areas needing precise robot movement.

Index Terms—Create3, A* Algorithm, Maze, Python, Pathfinding, Path Boundary approach, Left-wall-follow

I. INTRODUCTION

Autonomous robots have become increasingly prevalent in modern applications, from industrial automation to household tasks. A fundamental aspect of these robots is their ability to navigate through unknown environments, such as mazes, while avoiding obstacles and finding optimal paths. Maze-solving is a classic problem in robotics and artificial intelligence, often used to test navigation algorithms and sensor reliability. This project aimed to develop a maze-solving robot using the iRobot Create 3 platform. The key objective was to design and implement a reliable maze-solving algorithm that allows the robot to move autonomously from a predetermined start point to a goal point. The A* pathfinding algorithm was chosen for its efficiency and reliability in finding the shortest path in a grid-based environment. Additionally, the project involved integrating sensor data to dynamically adjust the robot's path and ensure successful navigation despite potential obstacles. A significant aspect of this project was using an image processing system to transform the maze image into a navigable matrix. This involved capturing the maze layout using a camera, processing the image to identify walls and paths, and converting this information into a format suitable for the A* algorithm. The image processing system was crucial in accurately mapping the maze and providing real-time updates to the robot's navigation system. This report

details the hardware setup, software implementation, and the results of the maze-solving algorithm. The methodology section describes the steps to transform the maze image into a navigable matrix, implementing the A* algorithm, and integrating sensor feedback for real-time path adjustments. The results section presents isolated and full system tests, the key findings, and observations from the testing phase, while the discussion analyses the strengths and limitations of the approach. Finally, the conclusion revisits the key findings and suggests potential areas for future improvement.

II. LITERATURE REVIEW

The iRobot Create 3 uses Python to solve tasks like navigating mazes. Python libraries like OpenCV play an important role by making pathfinding in maze images easy. Literature can discover the functionality of Create 3, how it is used in education and research, and what challenges the Create 3 could bring during this project. Furthermore, research was done on potential maze-solving algorithms and image processing using Python.

A. The iRobot Create 3

The iRobot Create 3 is a programmable robotic education, research, and prototyping platform. It builds on older iRobot models with significant improvements, including compatibility with ROS 2 (Robot Operating System 2), enhancing real-time communication and multi-robot system coordination. [1] Its hardware features include sensors for navigation, bump detection, and wheel encoders, along with connection options like Wi-Fi and Ethernet. The Create 3 is easily programmable through the iRobot Python API or using any IDEs (Integrated Development Environments) with Python and C++.

As seen in Fig. 1, the front of the Create 3 has a programmable multi-zone bumper to detect when the robot touches the wall, with seven infrared proximity sensors for obstacle detection. Using ROS 2 the three buttons at the top can also be overwritten. A ring of six RGB LEDs surrounds the power button.

Although the hardware is suitable for educational and research applications, its limitations—like the absence of advanced sensors such as LiDAR and a restricted payload capacity—make it less ideal for dynamic or outdoor environments. These constraints limit the Create 3's use to controlled indoor settings, where its educational potential is better realized.



Fig. 1: iRobot® Create® 3 Hardware Overview. Adapted from [1]

1) *Educational Applications:* The Create 3 can play a role in STEM education, offering hands-on learning opportunities. Studies show that using educational robots in classrooms enhances problem-solving and computational thinking among students, particularly in K-12 and university settings [2]. This is due to its programmability and ease of use. The iRobot Create 3 is a popular choice for robotics education. [12]

2) *Research Applications:* In research, Create 3's support for ROS 2 enables experiments in autonomous navigation, swarm robotics, and SLAM (Simultaneous Localization and Mapping) [3]. However, its limitations in sensor quality can affect the precision of research involving complex environments. The lack of LiDAR, for example, may restrict the effectiveness of high-precision navigation tasks.

3) *Challenges:* While the Create 3 is a strong platform for learning, it faces challenges. There are notable weaknesses of limited battery life, basic sensors, and a small payload capacity. These shortcomings make it best suited for indoor, controlled environments where such constraints are less critical [1]. The absence of advanced sensor systems, such as LiDAR, limits the robot's effectiveness in outdoor or dynamic settings.

Despite these limitations, the Create 3 remains a versatile tool, but sensor technology and battery performance improvements would expand its use in more complex environments.

B. Python Algorithms for Maze Solving

Path-finding is a crucial aspect of robotics, and game development, where the goal is to navigate from a starting point to a target while avoiding obstacles. [5] Python, with its many libraries, has become a popular language for implementing path-finding algorithms. The most common

algorithms for path-finding in Python include Dijkstra's algorithm and A* (A-star). [6]

1) The A* algorithm vs. the Dijkstra algorithm:

a) *Dijkstra's Algorithm:* Designed to find the shortest path in a weighted graph, Dijkstra's Algorithm explores all possible paths from the starting node, systematically expanding the closest undiscovered node until it reaches the destination. [7] This exhaustive approach guarantees the shortest path but can be inefficient, particularly in large graphs with many nodes, as it does not utilize heuristics to guide the search.

b) *A*:* On the other hand, A* enhances Dijkstra's approach by incorporating heuristics. It evaluates nodes based on the cost to reach them and an estimated cost to the target (using a heuristic function). This means A* can prioritize promising paths, significantly reducing the number of nodes it explores. As a result, A* often finds the shortest path faster than Dijkstra's algorithm, especially in real-time applications like gaming and robotics. [4] [8]

While both algorithms guarantee the shortest path, A* outperforms Dijkstra's in robotics applications where real-time path optimization is needed. Figure 2 shows an example of both algorithms.



Fig. 2: Image showing the difference between the A* (Left) and Dijkstra (Right) algorithms [9]

C. Image processing for maze solving using OpenCV

OpenCV is a widely used library for image processing and computer vision. It simplifies solving mazes by converting images to grayscale, applying thresholding, and using edge detection (e.g., Canny Edge Detection) to define maze boundaries. After mapping the walls using contour detection, path-finding algorithms like A* or Dijkstra's can be applied to navigate through the maze [10]. OpenCV's flexibility allows it to handle various maze designs, but more complex environments may require additional tuning for accurate results [11].

III. METHODOLOGY

In our maze-solving approach, we start by capturing an image of the maze and converting it into a matrix. We then use the A* algorithm to find the best path and set a boundary around it. The robot uses a left wall-following strategy to solve the maze. Whenever this approach nears

the boundary, its sensors are updated to create a virtual wall, guiding it back within the path's limits. The following sections explain each step of this methodology.

1) *Digital Mapping of the Maze and Start/End Point Detection*: First, we capture an image of the maze (see Fig. 5a) and correct any perspective distortions. We place four ArUco markers at each corner of the maze and use OpenCV's `warpPerspective` function to ensure the image is flat and properly aligned. The result is shown in Fig. 3b. After that, we use OpenCV's `matchTemplate` function to find the start and end points by matching small template images of these points to the main maze image.

Once the image is aligned, we apply edge detection to identify the maze walls. We use OpenCV's `GaussianBlur` to reduce noise, followed by `Canny` edge detection to highlight the walls. This process results in a binary matrix where 1 represents walls and 0 represents open paths, with the start and end points marked in Cartesian coordinates. The output of this process is shown in Fig. 3c. This matrix is then ready for the pathfinding step.

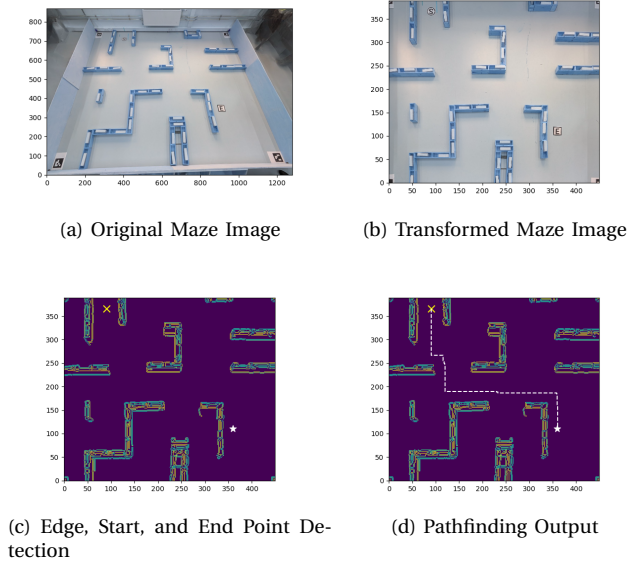


Fig. 3: Digital Mapping and Pathfinding Process: (a) Original maze image; (b) transformed image with corrected perspective; (c) edge detection showing start and end points; and (d) output of the optimal pathfinding step with the path from start to end marked.

2) *Finding the Optimal Path*: With the maze matrix prepared, we use the A* algorithm to find the shortest path from the start to the goal. The A* algorithm calculates a total cost for each position, called the f_score , which is a combination of two values:

- g_score : the distance traveled from the start point increased by 1 with each step.
- h_score : an estimate of the distance from the current position to the goal, calculated using the Manhattan

distance. This provides a simple measure of how many steps remain to reach the goal, considering only horizontal and vertical moves.

At each step, the algorithm evaluates possible moves and selects the position with the lowest f_score ($g_score + h_score$), continuing this process until it reaches the goal. During this process, the `is_valid` function checks each position to ensure it is within the maze, is not a wall, and maintains a safe buffer distance from walls. This buffer helps the robot avoid small gaps and provides enough space for smooth movement. The path resulting from this process is shown in Fig. 3d.

The algorithm also uses direction mapping to define possible movements (right, down, left, up) and to track the direction taken at each step. This information is stored in the `came_from` dictionary, allowing for accurate path reconstruction. Valid positions are added to an open list for evaluation, and `came_from` keeps track of each position's previous step and direction, enabling the algorithm to build the path to the goal. The final output is a list of coordinates for the robot to follow through the maze.

A. Navigation Through Physical and Virtual Walls

With the path determined, the robot navigates the maze using a left wall-following strategy. It follows the list of path coordinates, staying close to the left wall as it moves toward the goal. At each step, the robot checks its position to ensure it stays within the boundaries set by the A* algorithm.

If the robot gets close to the boundary in any direction, its sensors are updated to simulate a wall. This *virtual wall effect* helps keep the robot on the path by guiding it back. Adjusting the robot's sensor values to create virtual walls when needed avoids straying outside the path.

This approach combines the A* path with real-time sensor feedback. The left wall-following strategy keeps the robot moving along the path, while the virtual walls help it stay on course. This method allows the robot to follow a clear route from start to goal, responding to physical and virtual obstacles as it navigates the maze. The pseudocode for this navigation process is outlined below.

```
function navigate(robot, path):
    while not at_goal:
        if out_of_left_boundary: set left sensor high (wall)
        if out_of_right_boundary: set right sensor high (wall)
        if out_of_front_boundary: set front sensor high (wall)

        if left sensor detects space: turn left
        if left sensor too close: turn slightly right
        if right sensor too close: turn slightly left
        if front sensor detects obstacle: turn right
        if front sensor very close: turn sharply right

    if at_goal: stop and celebrate
```

For more details and access to the full implementation code, visit the project's GitHub repository.¹

¹The full code for this project is available at: <https://github.com/HanzeUAS-MasterSSE/project-introduction-to-sse-project-group-2>.

IV. RESULTS

A. Sensor Data Accuracy

As part of the evaluation, the robot's infrared proximity sensors were measured from three directions left, right, and front. We did not assess each sensor individually, as the initial assumption was that the sensor data would not be highly reliable. This assumption was confirmed during testing, as shown in Table I, which shows an example of the sensor's performance in detecting obstacles. The sensors produced inconsistent results when trying to establish any kind of function from the sensor readings as shown in Figure 4, where the function never truly reaches 0cm. The data revealed considerable variability, reinforcing the initial warning that the sensors might not provide the precision required for fine-grained obstacle detection.

CM	1	2	4	8	12	16	20
Front (Sensor 4)							
Sensor 1	20	21	19	14	15	9	9
Sensor 2	969	1007	508	175	63	29	19
Sensor 3	3737	3664	1301	367	273	216	150
Sensor 4	3663	3711	2992	947	389	207	119
Sensor 5	3281	2789	2166	660	274	144	86
Sensor 6	821	349	203	45	87	101	76
Sensor 7	17	22	15	10	10	5	7

TABLE I: Data when the iRobot Create 3 is facing a wall head on

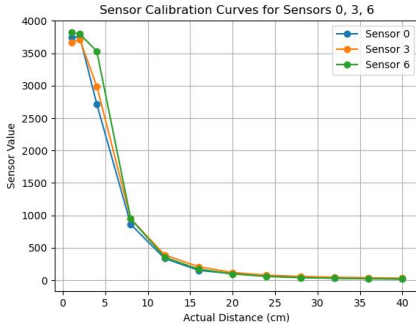


Fig. 4: Plotted data of 3 different sensors

These inaccuracies in sensor data did not significantly affect our results, as the project did not rely heavily on real-time obstacle detection for navigation. Instead, the maze-solving strategy primarily depended on preprocessed maze data and pathfinding algorithms. Nonetheless, the unreliable sensor data could impact the robot's performance in more dynamic, real-time environments.

B. Isolated System Tests

1) *Pathfinding Algorithm (A*)*: The A algorithm was tested in simulated maze environments to ensure that it consistently identified the shortest path from the start to the goal. In controlled conditions, the algorithm performed as expected, efficiently calculating the optimal path in the maze's binary matrix. However, when the results were

applied to the physical robot's navigation, some discrepancies were observed between the calculated path and the robot's actual movement. This was partly due to sensor inaccuracies and minor alignment issues with the maze.

2) *Image Processing with OpenCV*: The process of mapping the maze using OpenCV was largely successful in converting images of the maze into a usable binary matrix. As shown in Figure 5a, the system's edge detection and wall mapping worked well under optimal conditions. However, the "cleaned" path generated sometimes intersected walls, indicating that the maze processing was not perfect. These errors are primarily attributed to minor misalignments during the initial image capture or limitations in the contour detection algorithm, where noise and distortions occasionally caused incorrect wall recognition.

3) *Left Wall-Following Strategy*: The left wall-following strategy was tested separately to evaluate the robot's ability to adhere to this navigation approach. (See Figure 5d. and 5c) While the robot was generally able to follow the left wall, the strategy sometimes led to inefficient paths or dead ends in more complex maze structures. This behavior highlighted the limitations of the wall-following approach, which, while effective for simpler mazes, struggled in more intricate layouts.

C. Full System Tests

Once the system components were integrated, the full maze-solving system was tested in a controlled maze environment to observe how well the robot could execute the complete process—from image capture and pathfinding to physical navigation.

1) *Overall Performance*: The robot successfully navigated the maze in most cases, following the pre-calculated path and adjusting its movements based on sensor feedback when necessary. However, as indicated in Figure 6, the path provided to the robot sometimes included segments that passed too close to walls or, in some instances, even crossed through walls due to inaccuracies in the "clean path" algorithm. These issues were particularly evident in complex sections of the maze where OpenCV struggled to accurately detect walls or where minor alignment errors occurred during the image capture process.

2) *Success Rate*: The system demonstrated a roughly 70-80 percent success rate in solving mazes during the trials. The failures are primarily caused by navigation errors, where the robot deviates from the intended path or encounters obstacles that its sensors fail to detect. Despite these issues, in controlled environments where maze conditions were optimal, the system proved capable of solving the maze from start to finish.

D. Key Findings

1) *Sensor Data Accuracy*: The sensor data proved unreliable, confirming initial expectations. The inconsistencies, especially in detecting obstacles at certain angles or distances, highlight the limitations of relying on the Create 3's basic sensors for real-time navigation.

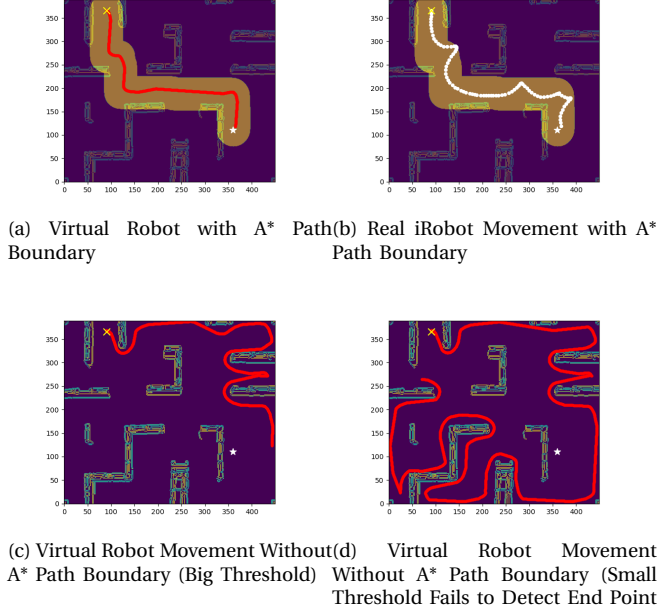


Fig. 5: Real and Virtual Robot Movements in a Maze Environment. (a) shows the virtual robot's movement with the A* path boundary, keeping the robot close to the optimal route. (b) illustrates the real iRobot's movement with the A* path boundary (transparent yellow region), where the robot adheres closely to the predefined path. (c) displays the virtual robot's movement without the A* path boundary, where a larger threshold is used for endpoint detection, leading to a wider, less accurate path. (d) presents the virtual robot's movement without the A* path boundary, where the small threshold fails to accurately detect the endpoint. The yellow "X" marks the start point, and the white star represents the goal.

2) *Pathfinding and Image Processing*: The A* algorithm was effective in digital simulations, but real-world implementation faced challenges due to inaccuracies in the maze's digital representation and the robot's physical capabilities. OpenCV's image processing was adequate for simple maze layouts but showed limitations in handling more complex or noisy environments, sometimes resulting in incorrect paths, as seen in Figure 6.

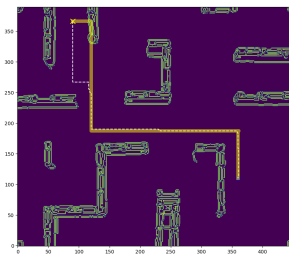


Fig. 6: An example of a cleaned path intersecting with a wall

3) *Navigation*: The left wall-following strategy worked well in simpler mazes but proved inefficient or problematic in more intricate layouts. The combination of pathfinding and real-time sensor feedback was sufficient for static mazes, though performance could degrade in dynamic or unpredictable environments.

V. DISCUSSION

The iRobot Create 3 proved to be an effective platform for solving mazes in controlled environments, though it faced limitations due to hardware and software constraints. The most significant issue encountered during testing was the accuracy of the robot's sensor data. As noted from Table I, the sensors, which were measured from the left, right, and front, confirmed their unreliability, as predicted. The sensors often provided inconsistent data, particularly when detecting obstacles at certain angles or distances. This highlights the limitations of relying solely on the Create 3's onboard sensors for real-time obstacle avoidance and accurate navigation.

Despite these sensor limitations, the core of the project—the integration of Python-based algorithms and image processing for pathfinding—performed reasonably well. The A* algorithm was successful in identifying the shortest path within the digital maze, but the translation of that path into the physical environment encountered challenges. Specifically, the image processing phase, which used OpenCV to generate a clean path for the robot to follow, occasionally produced inaccurate paths that intersected with walls, as shown in Figure 6. These issues were mostly due to limitations in OpenCV's edge detection and contouring capabilities in environments where noise or distortions affected wall detection.

Another key observation relates to the navigation strategy. The left wall-following method worked as a simple and reliable approach for basic mazes but struggled with more complex layouts where dead ends or inefficient paths reduced overall performance. Additionally, the real-time sensor updates, which were intended to correct deviations from the path, were hampered by the unreliability of the sensor data itself.

While the system demonstrated a reasonable success rate of 70-80 percent, its failures mostly stemmed from the robot deviating from the intended path or encountering obstacles it could not reliably detect. These challenges underscore the importance of considering both hardware limitations and the complexities of real-world environments when designing robotic navigation systems.

VI. CONCLUSION

This project was a great learning experience, giving us hands-on practice with robotics, programming, and problem-solving. By working with the iRobot Create 3 and Python algorithms to solve mazes, we learned a lot about how real robots work, how sensors can be unreliable, and how image processing can be used to guide a robot. The

challenges we faced, like inaccurate sensor data and problems with path creation, helped us better understand the difficulties in building systems for autonomous navigation.

Even though the iRobot Create 3 successfully solved mazes in simpler, controlled environments, we found several areas that could be improved. As expected, the robot's sensors were not very reliable for detecting obstacles, and errors in the image processing sometimes caused the robot to follow paths that went through walls. However, the A* algorithm worked well in simulations, and the overall system functioned as intended for basic maze-solving tasks, showing the potential of this platform for learning and research.

Several improvements could make the robot more effective. Adding better sensors, like LiDAR, would help the robot detect obstacles more accurately. Improving the image processing to reduce mistakes in path creation would also be helpful. Finally, exploring different ways for the robot to navigate mazes could improve its performance in more complex environments.

Overall, this project taught us much about robotics and the importance of problem-solving and system integration. It gave us a solid foundation for future work, where we can explore more advanced technologies to create even better and more reliable robotic systems.

VII. WHO DID WHAT

A general overview of which student contributed to which work (Table II)

Name	Responsibility
Soheil	Lead programmer responsible for developing the main code and utilities; focused on integrating various functionalities into the robot, including navigation and sensor data processing. Also wrote the methodology/implementation sections, detailing how the code was structured and executed
Rianne	Assisted in programming wall following and image processing; in charge of robot customization (LEDs, music, looks); wrote the literature review, results, discussion and conclusion; responsible for editing the report.
Amin	Researched the A* algorithm and explored basic Python concepts; gained an understanding of the basic programming aspects of the robot; wrote part of the research; authored the introduction and abstract.

TABLE II: Who Did What - A brief description of what each team member contributed to the project

ACKNOWLEDGMENT

We would like to thank our project tutor, Felipe Martins, for his support throughout this project. We also appreciate the project tutor from the other groups, Ewout Bergsma, and the other groups for their teamwork and shared experiences, which made our learning experience better.

REFERENCES

- [1] iRobot, "Create® 3 Documentation," iRobot Education, Accessed: Oct. 15, 2024. [Online]. Available: https://iroboteducation.github.io/create3_docs/

- [2] Ching, YH., Hsu, YC. "Educational Robotics for Developing Computational Thinking in Young Learners: A Systematic Review". TechTrends 68, 423-434 (2024). <https://doi.org/10.1007/s11528-023-00841-1>
- [3] Y. Chen et al., "Trajectory-Based SLAM for Indoor Mobile Robots with Limited Sensing Capabilities," 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 2023, pp. 9147-9154, doi: 10.1109/IROS55552.2023.10341518.
- [4] K. Talan and G. R. Bamnote, "Shortest path finding using A* algorithm and minimum weight node first principle," International Journal of Innovative Research in Computer and Communication Engineering, vol. 3, no. 5, pp. 4290-4295, May 2015. DOI: 10.15680/ijirce.2015.0305131.
- [5] Z. A. Algfoor, M. S. Sunar, and H. Kolivand, "A comprehensive study on pathfinding techniques for robotics and video games," Journal of Computer Science and Technology, vol. 2015, Apr. 2015. doi: 10.1155/2015/736138.
- [6] A. Bresser, "Pathfinding: A pathfinding algorithm library for Python," PyPI, License: MIT. [Online]. Available: <https://pypi.org/project/pathfinding/>. [Accessed: 15-Oct-2024].
- [7] M. A. Javaid, "Understanding Dijkstra's algorithm," SSRN, Apr. 2013. [Online]. Available: <https://ssrn.com/abstract=2340905>. doi: 10.2139/ssrn.2340905.
- [8] P. Benjamin, "A* algorithm," Pace University, [Online]. Available: <https://csis.pace.edu/~benjamin/teaching/cs627/webfiles/Astar.pdf>. [Accessed: 15-Oct-2024].
- [9] K. Wang, "Compare A* with Dijkstra algorithm," Plain As Life, Feb. 28, 2015. [Online]. Available: <https://plainaslife.blogspot.com/2015/02/compare-with-dijkstra-algorithm.html>. [Accessed: Oct. 16, 2024].
- [10] G. Bradski and A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library. Sebastopol, CA: O'Reilly Media, 2008.
- [11] A. Srivastava, OpenCV with Python by Example. Birmingham, UK: Packt Publishing, 2016.
- [12] P. Allcock, "iRobot launches the Create 3 educational robot, based on the Roomba i3 series," NotebookCheck, Apr. 5, 2022. [Online]. Available: <https://www.notebookcheck.net/iRobot-launches-the-Create-3-educational-robot-based-on-the-Roomba-i3-series.622067.0.html>. [Accessed: Oct. 16, 2024].